

Compilador BRL — Documentação Formal

Trabalho Prático — Compiladores

Ciência da Computação — Dom Helder / Escola Superior

Prof. Dr. Marcos W. Rodrigues

Integrantes do Grupo:

João Gabriel Ribeiro Holanda

João Victor Fernandes Lima

Luan Tadeu Lima Rezende Dias

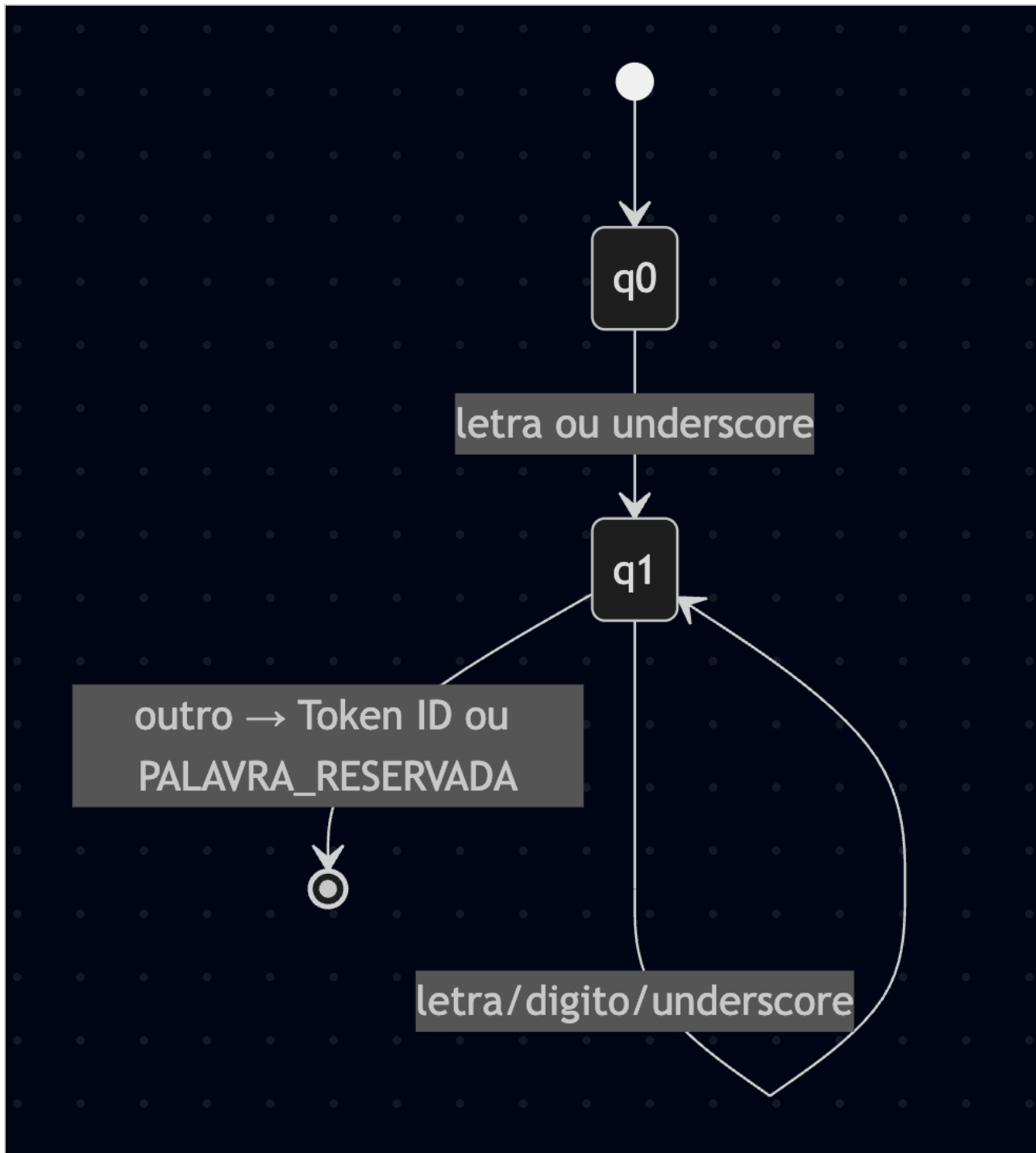
I — Autômato Finito Determinístico (AFD)

O AFD do analisador léxico é apresentado em subdiagramas para melhor legibilidade. Todos os subdiagramas partem do estado inicial **q0** e representam, juntos, o autômato completo.

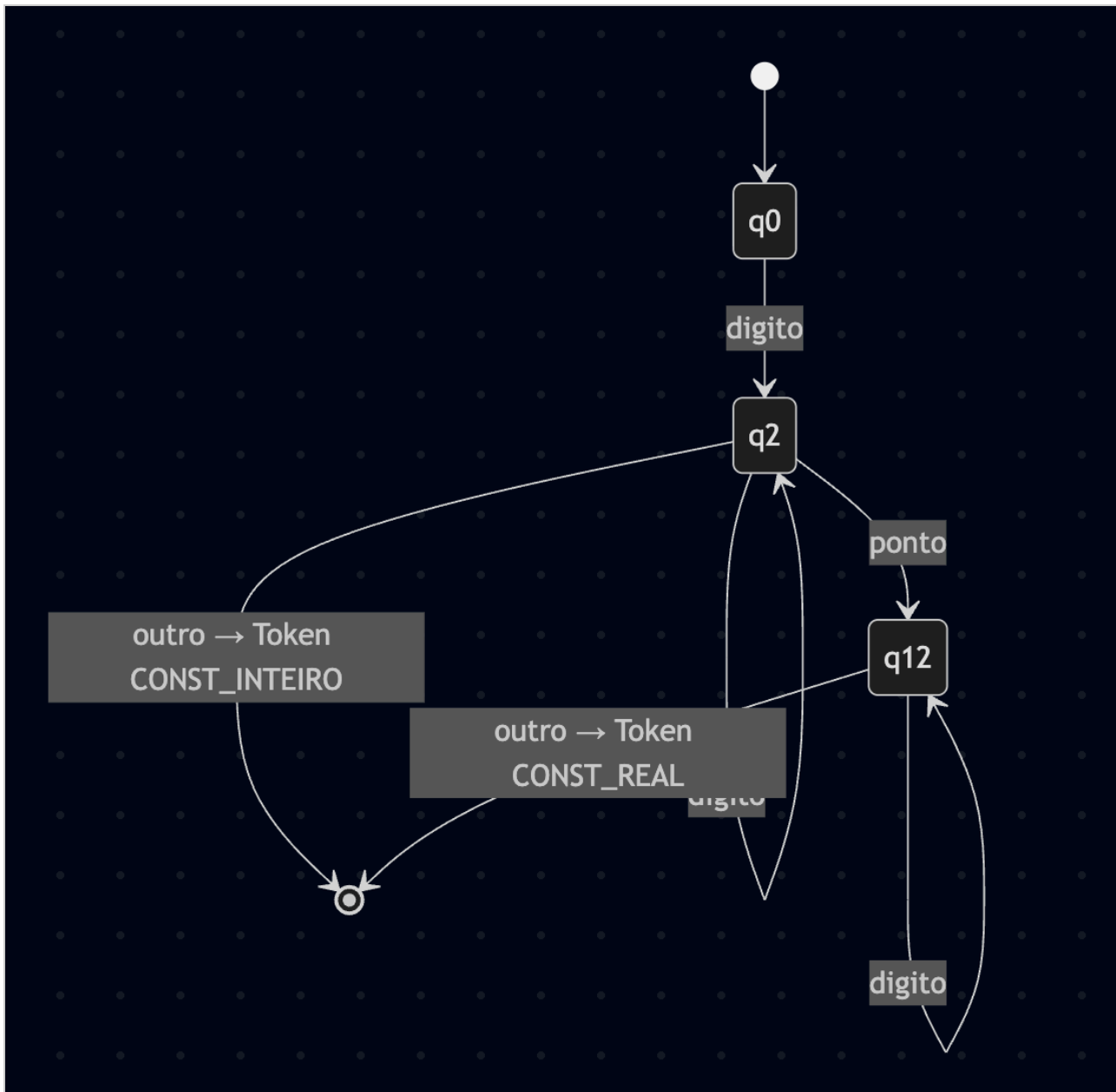
Definição formal: $M = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{q_0, q_1, q_2, q_3, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}, q_{12}, q_{13}\} \cup \text{estados finais}$
- $\Sigma = \{\text{letra, dígito, sublinhado, espaço, +, -, *, /, <, >, =, \&, :, ;, ,, (,), ", .}\} \cup \{\text{CR, LF}\}$
- **q0** = estado inicial
- **F** = estados de aceitação (cada token reconhecido)

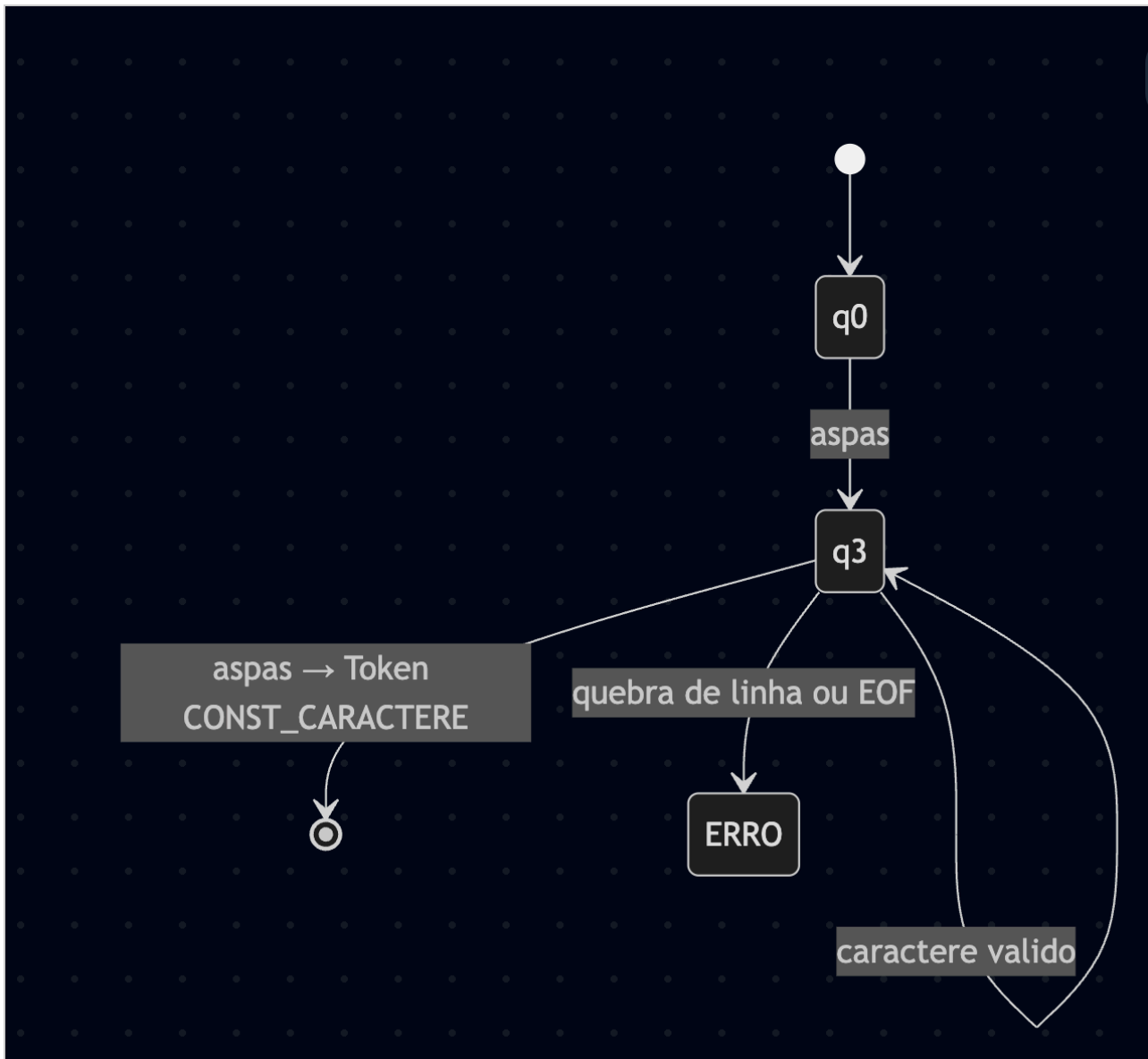
1. Identificadores e Palavras Reservadas



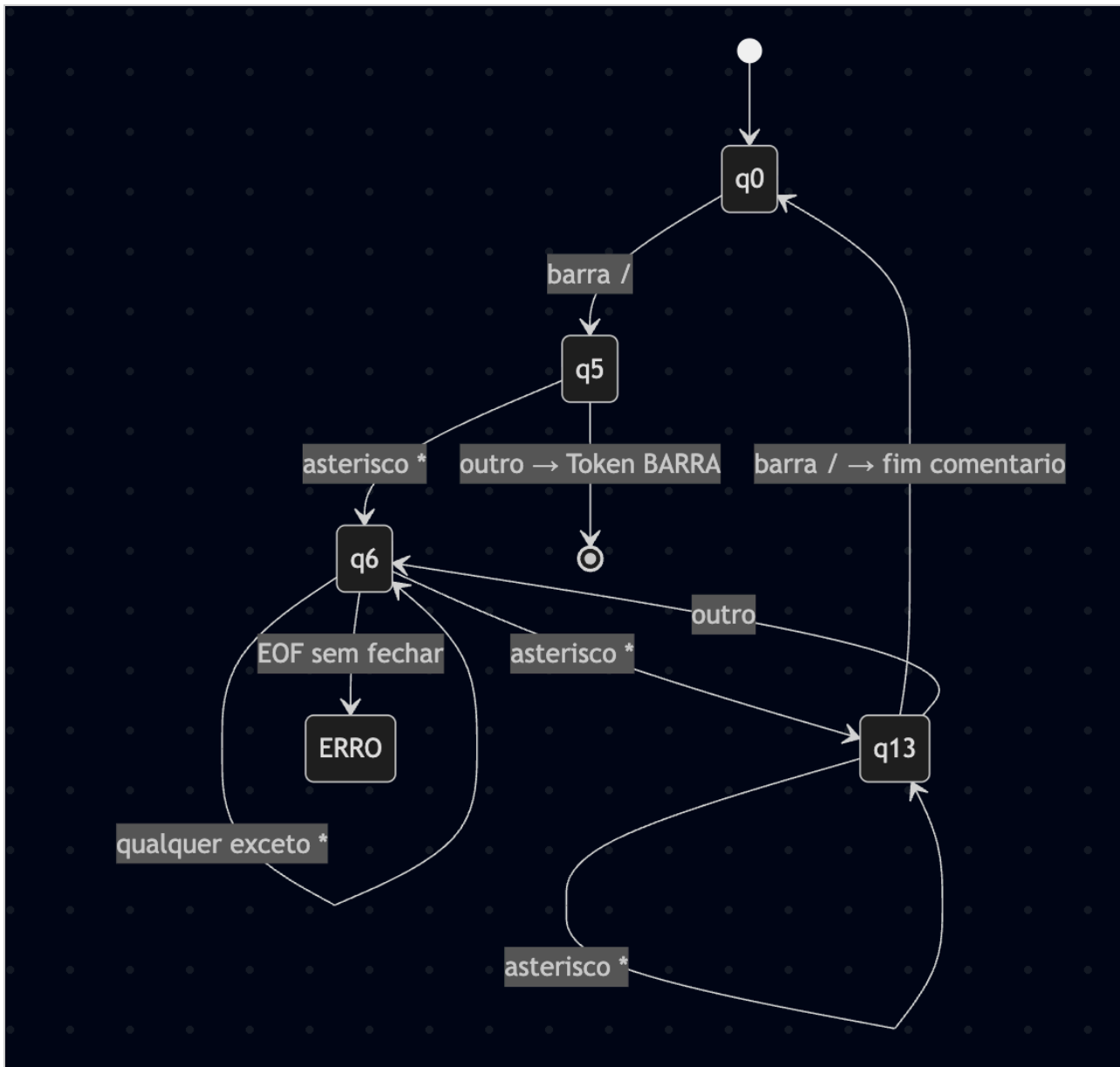
2. Constantes Numéricas (Inteiro e Real)



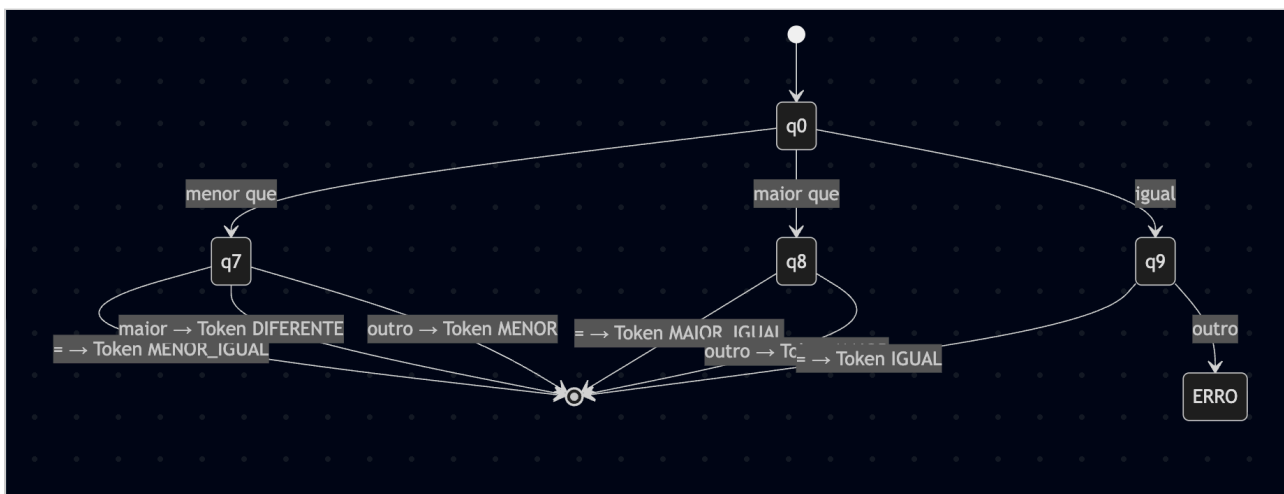
3. Constantes Caractere (Strings)



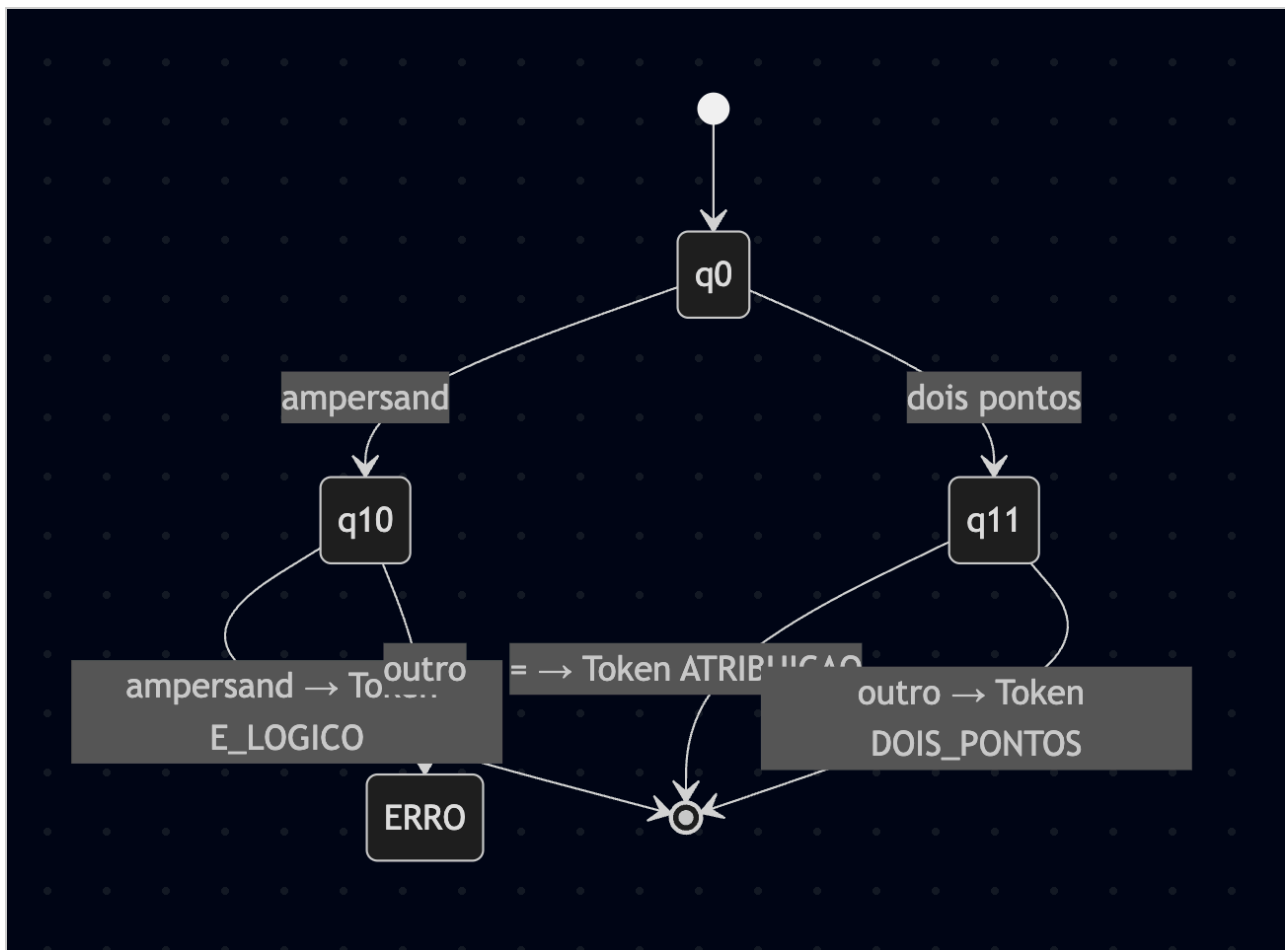
4. Comentários



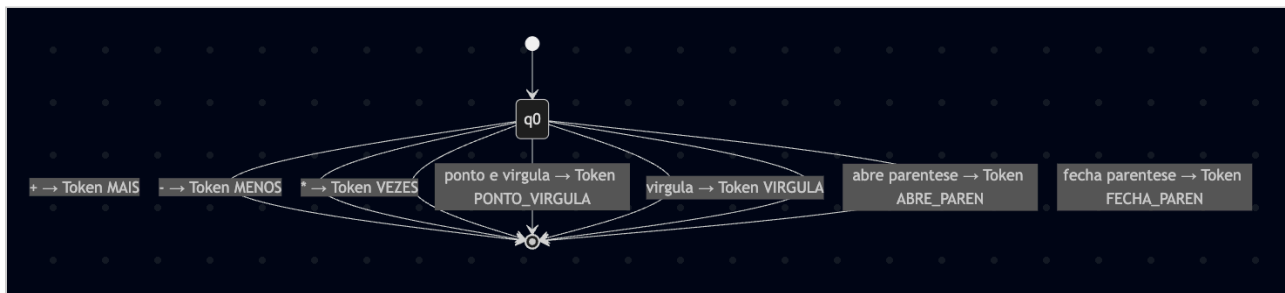
5. Operadores Relacionais



6. Operadores Lógicos e Atribuição



7. Delimitadores e Operadores Simples



Palavras Reservadas

Palavra	Token
inteiro	INTEIRO
caractere	CARACTERE
logico	LOGICO
real	REAL
se	SE

entao	ENTAO
senao	SENAO
enquanto	ENQUANTO
faca	FACA
leitura	LEITURA
escrita	ESCRITA
ou	OU
e	E
nao	NAO
inicio	INICIO
fim	FIM
div	DIV
mod	MOD
verdadeiro	VERDADEIRO
falso	FALSO

II — Gramática Formal do Compilador BRL

2.1 — Gramática Livre de Contexto (BNF)

```

<programa>      → 'inicio' <identificador> ';' <declaracoes> <instrucoes>
'fim'

<declaracoes>   → <declaracao> ';' <declaracoes>
                  | ε

<declaracao>    → <id_list> ':' <tipo>

<id_list>       → <identificador> ',' <id_list>
                  | <identificador>

<tipo>          → 'inteiro'
                  | 'caractere'
                  | 'logico'
                  | 'real'

<instrucoes>    → <instrucao> ';' <instrucoes>
                  | <instrucao>
                  | ε

<instrucao>     → <atribuicao>
                  | <se>
                  | <enquanto>
                  | <leitura>
                  | <escrita>

<atribuicao>     → <identificador> ':=' <expressao>

<se>            → 'se' <expressao> 'entao' 'inicio' <instrucoes> 'fim'
                  | 'se' <expressao> 'entao' 'inicio' <instrucoes> 'fim'
                  | 'senao' 'inicio' <instrucoes> 'fim'

<enquanto>      → 'enquanto' <expressao> 'faca' 'inicio' <instrucoes> 'fim'

<leitura>       → 'leitura' '(' <id_list> ')'

<escrita>       → 'escrita' '(' <expressao> ')'

<expressao>     → <exp_simples> <relop> <exp_simples>
                  | <exp_simples>

<exp_simples>   → <termo> <addop> <exp_simples>

```



```

| <termo>

<termo>      → <fator> <mulop> <termo>
| <fator>

<fator>      → <identificador>
| <constante>
| '(' <expressao> ')'
| 'nao' <fator>
| '+' <fator>
| '-' <fator>

<constante>  → <const_inteiro>
| <const_real>
| <const_caractere>
| 'verdadeiro'
| 'falso'

<relop>      → '==' | '<>' | '<' | '>' | '<=' | '>='

<addop>      → '+' | '-' | 'ou'

<mulop>      → '*' | '/' | 'div' | 'mod' | '&&'

```

2.2 — Definições Léxicas

```

<identificador> → (letra | '_' ) (letra | dígito | '_' ) *
<const_inteiro> → dígito +
<const_real>    → dígito + '.' dígito +
<const_caractere> → "'" (caractere_válido) * "'"
<letra>         → [a-zA-Z]
<dígito>        → [0-9]
<caractere_válido> → qualquer caractere exceto "'", CR e LF

```

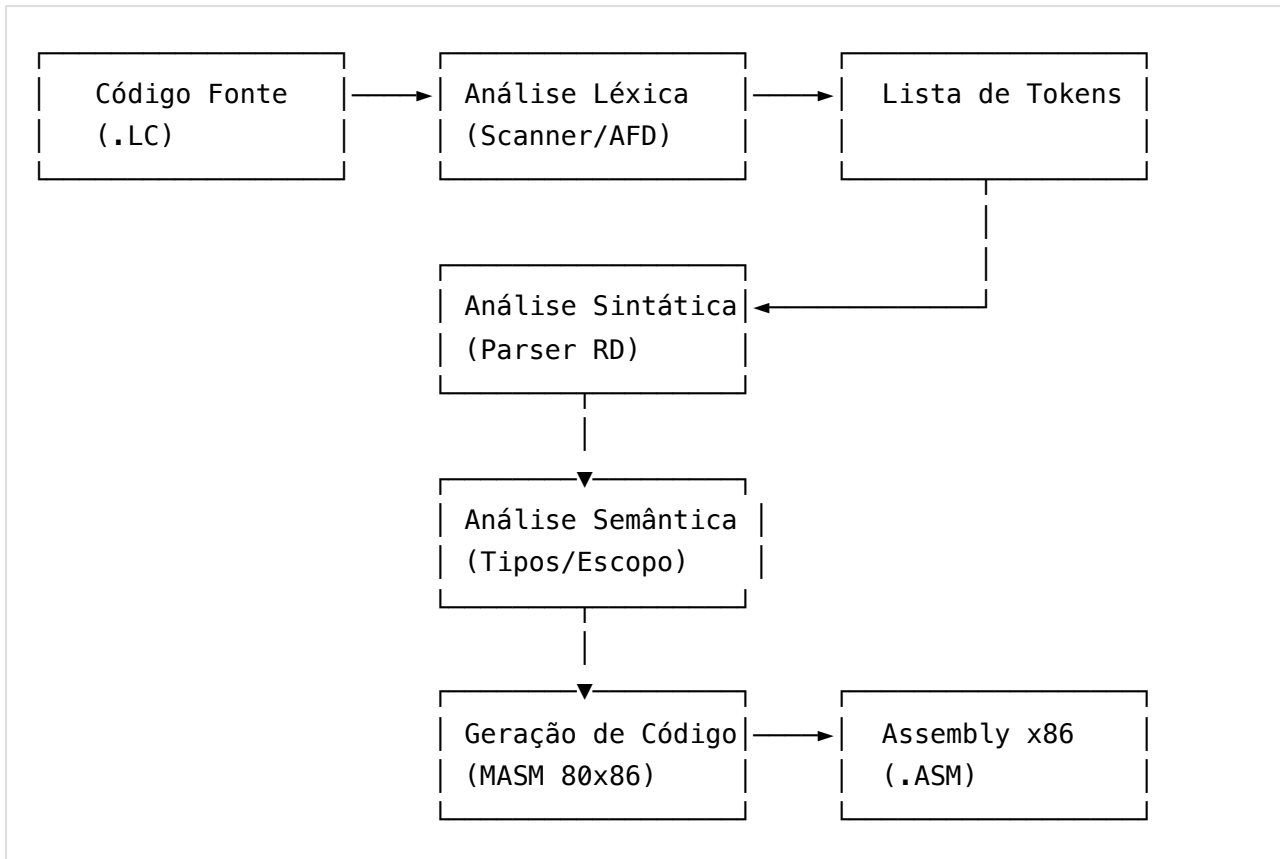
2.3 — Precedência de Operadores (da maior para a menor)

Prioridade	Operadores	Associatividade
1	()	—
2	nao, + (unário), - (unário)	Direita
3	*, /, div, mod, &&	Esquerda
4	+, -, ou	Esquerda
5	==, <>, <, >, <=, >=	Esquerda

2.4 — Regras Semânticas

Regra	Descrição
S1	Todo identificador deve ser declarado antes do uso
S2	Não pode haver declaração duplicada do mesmo identificador
S3	Atribuição: tipo da expressão deve ser compatível com tipo da variável
S4	Promoção de tipo: inteiro pode ser atribuído a real
S5	Operadores aritméticos (+, -, *, /) exigem operandos numéricos
S6	Operadores div e mod exigem operandos inteiros
S7	Operadores lógicos (&&, ou, nao) exigem operandos lógicos
S8	Operadores relacionais exigem operandos numéricos (exceto == entre caracteres)
S9	Concatenação (+) entre caracteres resulta em caractere
S10	Condição de 'se' e 'enquanto' deve ser do tipo lógico

III — Arquitetura do Compilador



Front-end:

- `AnalizadorLexico.java` — Scanner baseado no AFD
- `AnalizadorSintatico.java` — Parser descendente recursivo
- `AnalizadorSemantico.java` — Verificação de tipos e declarações

Back-end:

- `GeradorCodigo.java` — Geração de Assembly MASM x86

Estruturas auxiliares:

- `Token.java` / `TokenType.java` — Representação de tokens
- `TabelaSimbolos.java` — Tabela de símbolos com endereços de memória